Lab 08: Dynamic Web Pages with Selenium

INFO 4617/5617 · Week 8 · Chapter 8, Dynamic Web Pages with Selenium

Today we code together. Each step below says what we're building and why, and its code cell holds only the comments that frame it: we write the code live, in class. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

We'll work on the chapter's practice site, Quotes to Scrape, which serves the same quotes as plain HTML, as a script, through a hidden API, behind a ten-second delay, as an infinite scroll, and behind a login. First we check with `requests` whether a page needs a browser at all; then we drive Chrome from Python, headless, to find, click, type, wait, scroll, and log in; and last we do the same with a Playwright script.

Before class: finish week 8's setup handout, so that Chrome opened from chapter 8's notebook. Open this notebook in the same environment (`webdata`, or Anaconda's own). Have your email address ready for the User-Agent. Step 10 needs Playwright's browsers, about 320 MB: if you want to run it yourself, run `python -m playwright install chromium` at home first.

On Friday, run every cell above where we stopped (**Run → Run All Above Selected Cell**).

> **Instructor key.** Every cell is filled in and was run against the live sites on 2026-10-21, headless, from the textbook at commit `d9306b7`, except the last cell of Step 10 (below). The quotes sandbox's answers (counts, authors, the login's 10 links) will match a rerun; xkcd's front-page comic, the random comic, Chrome's version, and every timing will not. The key ran on a Linux server as root, where Chrome won't start without `--no-sandbox` (the chapter's "Headless Mode", "On a server or in a container"), so the run set the environment variable `LAB_CHROME_FLAGS` to `--no-sandbox --disable-dev-shm-usage`, which the last two lines of Step 4's first cell read. Those two lines are for the key only: don't type them in class, and on a laptop they change nothing. Step 9 read the practice password from `QUOTES_PASSWORD`. Step 10's script was written, but running it is tagged `not-run-live`: the server's Playwright library (1.64) found no browser of its own version, and the server doesn't download browsers. The same script, pointed at the server's older Playwright Chromium, printed `Albert Einstein` and `Page 2: 10 quotes`. Run that cell before class.

Step 0 — Set up (5 minutes)

Import what we'll use first, and define the User-Agent that our `requests` calls send. The second cell is given: it is step 1 of the chapter's Before the First Browser, which you ran in the setup handout. Selenium Manager reads its settings from environment variables each time it runs, and they last only until the kernel restarts, so they're set before anything starts a browser. `SE_SKIP_DRIVER_IN_PATH` makes it ignore stray drivers on your `PATH`; `SE_AVOID_STATS` stops the usage statistics it otherwise sends.

```
# Import json, os, re, time, pandas (as pd), requests, and BeautifulSoup
# Define HEADERS with the User-Agent "WebDataScience/1.0 (your-email@colorado.edu)", with your
 ↪  email
import json
import os
import re
import time

import pandas as pd
import requests
from bs4 import BeautifulSoup

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
```

```
# Given: Selenium Manager's settings, from the chapter's "Before the First Browser", step 1
import sys
from pathlib import Path

import selenium

manager = Path(sys.prefix) / ("Scripts/selenium-manager.exe" if os.name == "nt" else
 ↪  "bin/selenium-manager")
if "SE_MANAGER_PATH" not in os.environ and manager.is_file():
    os.environ["SE_MANAGER_PATH"] = str(manager)
os.environ["SE_SKIP_DRIVER_IN_PATH"] = "true"  # Ignore stray drivers on your PATH
os.environ["SE_AVOID_STATS"] = "true"          # Send no usage statistics
print(f"selenium {selenium.__version__}; Selenium Manager: {os.environ.get('SE_MANAGER_PATH',
 ↪  'inside the selenium package')}")
```

```
selenium 4.51.0; Selenium Manager: inside the selenium package
```

💬 Discuss

- In week 2 we wrote a User-Agent that names us and how to reach us. Chrome sends its own. Who will the sites in today's lab think is visiting when Selenium drives it?
- Selenium Manager sends usage statistics unless `SE_AVOID_STATS` is set. What does it learn about you, and would you have known without reading the chapter?
- The setup handout checked what Selenium Manager found before it started the first browser. Why check first, rather than read the error when `webdriver.Chrome()` fails?

Step 1 — What `requests` sees (5 minutes)

The chapter's first check is by hand: open https://quotes.toscrape.com/js/, right-click, and choose **View Page Source**. Ten quotes are on the screen, but there's no `<div class="quote">` in the source. Then turn JavaScript off (**Command Menu**, Control+Shift+P or Command+Shift+P, type `javascript`, **Disable JavaScript**) and reload: the quotes vanish (Do You Need a Browser?, Checks 1 and 2).

Now the same check in code. Fetch the page with `requests` and count `div.quote` with BeautifulSoup. Then look where View Source showed the quotes: a `<script>` holds them as a JavaScript array named `data`. Cut it out with a regular expression (`re.search`, with `re.DOTALL` so `.` matches line breaks), and parse it with `json.loads()`, because it is written in JSON syntax.

```
# Fetch https://quotes.toscrape.com/js/ with HEADERS; print the status and how many div.quote
 ↪  elements BeautifulSoup finds
# Cut the array out of the script: re.search(r"var data = (\[.*?\]);", response.text, re.DOTALL)
# Parse group 1 with json.loads(); print how many quotes it holds and the first one's author
response = requests.get("https://quotes.toscrape.com/js/", headers=HEADERS)
soup = BeautifulSoup(response.text, "html.parser")
print(f"{response.status_code}: {len(soup.select('div.quote'))} div.quote elements")
match = re.search(r"var data = (\[.*?\]);", response.text, re.DOTALL)
quotes = json.loads(match.group(1))
print(f"{len(quotes)} quotes in the script; the first is by {quotes[0]['author']['name']}")
```

```
200: 0 div.quote elements
10 quotes in the script; the first is by Albert Einstein
```

💬 Discuss

- `requests` got a 200 and found zero quotes, with no error. In week 6, IMDb answered with a 202 and zero characters. How would you notice either failure in a loop over a thousand pages?
- The quotes were in the response all along. When is cutting them out of a script better than starting a browser, and what could break it?

- Week 4's `json.loads()` parsed the array because it happens to be written in JSON syntax. What would happen if the page wrote its keys without quotation marks, as JavaScript allows?

Step 2 — The hidden API, two pages by hand (6 minutes)

The chapter's third check, by hand first: open https://quotes.toscrape.com/scroll, open developer tools' **Network** tab, click **Fetch/XHR**, and scroll. Each time you near the bottom, a request appears: `quotes?page=2`, `quotes?page=3`. Click one: its **Preview** is JSON, ten quotes and a `has_next` flag. This is a **hidden API**: nobody documents it, but everything you need to call it is in the Network tab (Check 3: Watch the Network Tab).

Test one request before writing any loop: pass `params={"page": 1}` to `requests.get()` and look at `response.url`, the `Content-Type`, and the JSON's keys. Then ask for page 2 the same way, changing only the parameter.

```
# Request https://quotes.toscrape.com/api/quotes with params={"page": 1} and HEADERS; print the
 ↪  URL and the Content-Type
# Read the JSON into data; print its keys, then its page, has_next, and number of quotes
response = requests.get("https://quotes.toscrape.com/api/quotes", params={"page": 1},
 ↪  headers=HEADERS)
print(f"{response.url}: {response.headers['Content-Type']}")
data = response.json()
print(list(data))
print(f"Page {data['page']}: has_next {data['has_next']}, {len(data['quotes'])} quotes")
```

```
https://quotes.toscrape.com/api/quotes?page=1: application/json
['has_next', 'page', 'quotes', 'tag', 'top_ten_tags']
Page 1: has_next True, 10 quotes
```

```
# Request page 2 the same way; print its page, has_next, number of quotes, and first author
response = requests.get("https://quotes.toscrape.com/api/quotes", params={"page": 2},
 ↪  headers=HEADERS)
data = response.json()
print(f"Page {data['page']}: has_next {data['has_next']}, {len(data['quotes'])} quotes, first by
 ↪  {data['quotes'][0]['author']['name']}")
```

```
Page 2: has_next True, 10 quotes, first by Marilyn Monroe
```

💬 Discuss

- We asked for page 1 and then page 2 before writing any loop. What did the second request tell us that the first couldn't?
- In week 5, **Copy as cURL** showed every header the browser sent. If this API answered the browser with 200 and you with 403, what would you compare first?
- Nobody promised to keep `/api/quotes` working. Week 7 found that 38% of 2013's web pages were gone a decade later. What should you do with what you collect from it?

Step 3 — Every page: a function, then the whole loop (10 minutes)

Gather Step 2's two lines into `get_page()`, which takes a page number and returns that page's JSON, and use it for the chapter's two probes: the last page, and one page past the end ("Test one request").

The loop that follows `has_next` to the end is given: `get_all_quotes()` asks for page 1, 2, 3, … until `has_next` is false, and pauses a second between pages. Read it together and compare it with `get_page()`: the request is the same; the loop adds the page counter, the list, the stop, and the pause. It also takes a `tag`, the parameter the replies hint at with `"tag": null`; `requests` leaves a parameter out when its value is `None`. Use it to collect every quote, time it, and make a DataFrame with one row per quote, as the chapter does (Write the scraper).

```
# Define get_page(page): request /api/quotes with params={"page": page} and HEADERS, and return
 ↪  the JSON
# Use it for page 10 and page 11; print each one's has_next and number of quotes
def get_page(page):
    response = requests.get("https://quotes.toscrape.com/api/quotes", params={"page": page},
 ↪  headers=HEADERS)
    return response.json()

page_10 = get_page(10)
page_11 = get_page(11)
print(f"Page 10: has_next {page_10['has_next']}, {len(page_10['quotes'])} quotes")
print(f"Page 11: has_next {page_11['has_next']}, {len(page_11['quotes'])} quotes")
```

```
Page 10: has_next False, 10 quotes
Page 11: has_next False, 0 quotes
```

```
# Given: follow has_next from page 1 to the end, a second apart. Compare it with get_page().
def get_all_quotes(tag=None):
    quotes = []
    page = 1
    while True:
        response = requests.get("https://quotes.toscrape.com/api/quotes",
                                params={"page": page, "tag": tag}, headers=HEADERS)
        data = response.json()
        quotes.extend(data["quotes"])
        if not data["has_next"]:
            return quotes
        page += 1
        time.sleep(1)
```

```
# Time get_all_quotes(); make api_quotes, a DataFrame with each quote's author (the name), text,
 ↪  and tags (joined with ", ")
# Print its shape and the seconds it took
start = time.time()
all_quotes = get_all_quotes()
api_seconds = time.time() - start
api_quotes = pd.DataFrame([
    {"author": q["author"]["name"], "text": q["text"], "tags": ", ".join(q["tags"])}
    for q in all_quotes
])
print(f"{api_quotes.shape} in {api_seconds:.1f} seconds")
```

```
(100, 3) in 14.1 seconds
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: every reply carries `top_ten_tags`, the site's own count of its most-used tags. Pick one, collect its quotes with `get_all_quotes(tag=...)`, and check the API's count against what came back. **Share:** your tag, how many quotes came back, and whether that matches `top_ten_tags`.

```
# Microsprint: make top_ten = dict(page_10["top_ten_tags"]) and print it; pick a tag
# Collect its quotes with get_all_quotes(tag=...); print how many came back, and what top_ten
 ↪  says
top_ten = dict(page_10["top_ten_tags"])
print(top_ten)
tagged = get_all_quotes(tag="humor")
print(f"humor: {len(tagged)} quotes; top_ten_tags says {top_ten['humor']}")
```

```
{'love': 14, 'inspirational': 13, 'life': 13, 'humor': 12, 'books': 11, 'reading': 7,
 ↪  'friendship': 5, 'friends': 4, 'truth': 4, 'simile': 3}
```

```
humor: 12 quotes; top_ten_tags says 12
```

💬 Discuss

- Page 11 answered with an empty list and no error. What would a loop that stopped only on an error have done, and what does `get_all_quotes()` stop on instead?
- `get_all_quotes()` assumes every reply is JSON with a `has_next`. What would break if the site answered a 429, or an HTML error page, and what does the chapter's loop add to stop on one?
- A tag is a filter someone else chose. What would a study of "humor" in these quotes miss by trusting it?

Step 4 — Start a headless browser (6 minutes)

When all three checks come up empty, you need a browser. Chrome's `Options` say how to start it: `--headless=new` runs it with no window, and `--window-size=1280,800` lays pages out as a laptop window would, because without it headless Chrome lays them out 780 pixels wide (Headless Mode). Build `options` once: every browser in this lab starts from it, so it is the one place to switch headless off. On your laptop, leave out the `--headless=new` line and a Chrome window opens instead, labelled Chrome for Testing if Selenium Manager downloaded it; leave it alone while the code drives it. Watching it is how you catch a click that lands on the wrong button.

Then start Chrome with `webdriver.Chrome(options=options)`, timing it, load the `/js/` page, and save a screenshot: with no window, a screenshot is how you see what the browser saw.

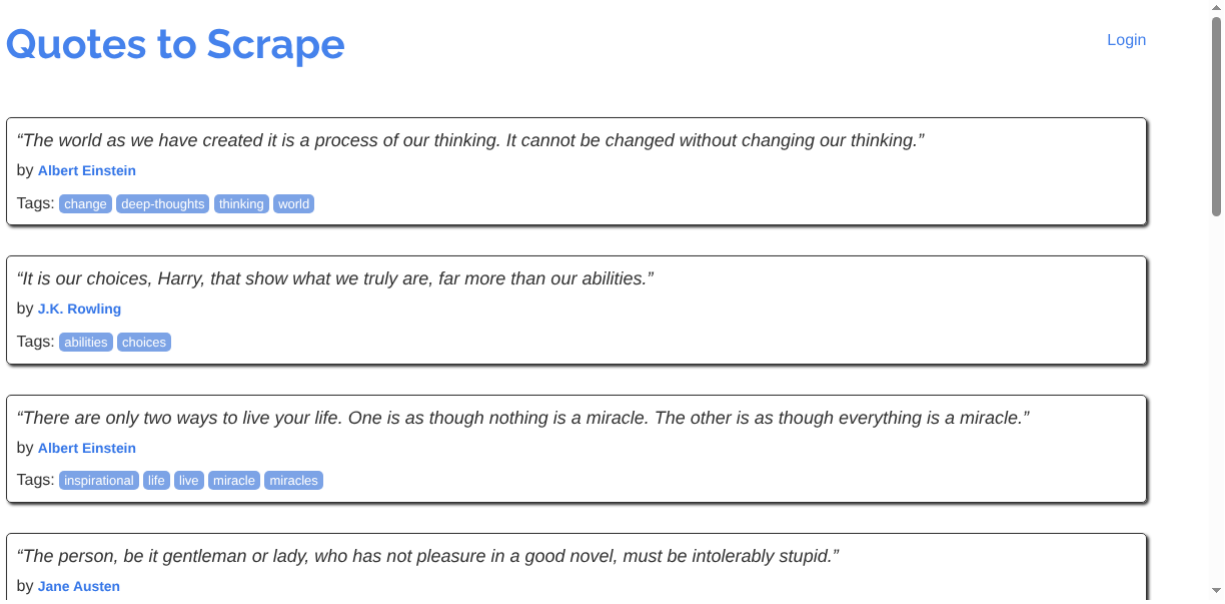
```
# Import webdriver, Options, and By from selenium, and Image from IPython.display
# Build options: Options(), then add "--headless=new" (leave it out on your laptop) and
 ↪  "--window-size=1280,800"
from IPython.display import Image
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By

options = Options()
options.add_argument("--headless=new")
options.add_argument("--window-size=1280,800")
# Key only, not typed in class: the key's server runs Chrome as root, which needs --no-sandbox
 ↪  ("Headless Mode")
for flag in os.environ.get("LAB_CHROME_FLAGS", "").split():
    options.add_argument(flag)
```

```
# Start driver = webdriver.Chrome(options=options), timing it; print the seconds and
 ↪  driver.capabilities["browserVersion"]
# Load https://quotes.toscrape.com/js/ and print how many div.quote elements it has
# Save a screenshot as quotes-headless.png, and show it with Image(..., width=640)
start = time.time()
driver = webdriver.Chrome(options=options)
print(f"Started in {time.time() - start:.1f} s: Chrome {driver.capabilities['browserVersion']}")
driver.get("https://quotes.toscrape.com/js/")
print(f"{len(driver.find_elements(By.CSS_SELECTOR, 'div.quote'))} div.quote elements")
driver.save_screenshot("quotes-headless.png")
Image("quotes-headless.png", width=640)
```

```
Started in 16.2 s: Chrome 156.0.8078.47
```

```
10 div.quote elements
```



💬 Discuss

- Headless Chrome names itself `HeadlessChrome` in its User-Agent, and every browser Selenium drives tells pages that `navigator.webdriver` is `true`. Week 5 called rewriting a User-Agent spoofing. Should a research scraper hide either one?
- The key adds `--no-sandbox` because it runs as root on a server. Why does the chapter tell you to leave the sandbox on, on your own laptop, even when Chrome won't start?
- What can the screenshot tell you that the count of quotes can't?

Step 5 — Find elements, and read them (10 minutes)

`find_element()` takes a **locator**: a strategy from `By` and a value. On xkcd's front page, try five of the eight strategies the chapter tabulates: `By.CLASS_NAME` for the two navigation rows, `By.CSS_SELECTOR` for their links, `By.LINK_TEXT` and `By.PARTIAL_LINK_TEXT` for the **Random** and **< Prev** links, and `By.XPATH` to step up from the comic to its parent with `/..` (The Eight Strategies). `find_element()` returns the first match; `find_elements()` returns a list, empty when there's none.

To search inside an element, call the method on the element, and start an XPath with a dot: without it, `//` searches the whole page (”One Element or Many”). And Selenium reads the page as the browser shows it: xkcd keeps a transcript of many comics in a `<div id="transcript">` that its style hides, so `.text` returns nothing, while `get_attribute("textContent")`, like BeautifulSoup's `get_text()`, returns what the markup holds (”How This Differs from BeautifulSoup's `find_all()`”).

```
# Load https://xkcd.com; print how many elements have class "comicNav", and the first five links'
 ↪  text (CSS "ul.comicNav a")
# Print the Random link's href (By.LINK_TEXT), the "Prev" link's text (By.PARTIAL_LINK_TEXT),
#   and the id of div#comic's parent (By.XPATH "//div[@id='comic']/..")
driver.get("https://xkcd.com")
links = driver.find_elements(By.CSS_SELECTOR, "ul.comicNav a")
print(f"{len(driver.find_elements(By.CLASS_NAME, 'comicNav'))} comicNav rows; {len(links)} links:
 ↪  {[a.text for a in links[:5]]}")
print(driver.find_element(By.LINK_TEXT, "Random").get_attribute("href"))
print(driver.find_element(By.PARTIAL_LINK_TEXT, "Prev").text)
print(driver.find_element(By.XPATH, "//div[@id='comic']/..").get_attribute("id"))
```

```
2 comicNav rows; 10 links: ['|<', '< Prev', 'Random', 'Next >', '>|']
https://c.xkcd.com/random/comic/
< Prev
```

```
middleContainer
```

```
# Inside the first comicNav, count the links with XPath ".//a", then with "//a"
# Load https://xkcd.com/353/; print repr() of div#transcript's .text, then the first 59
 ↪  characters of its textContent
nav = driver.find_element(By.CLASS_NAME, "comicNav")
print(f"{len(nav.find_elements(By.XPATH, './/a'))} links with .//a;
 ↪  {len(nav.find_elements(By.XPATH, '//a'))} with //a")
driver.get("https://xkcd.com/353/")  # The comic "Python"
transcript = driver.find_element(By.ID, "transcript")
print(repr(transcript.text))
print(transcript.get_attribute("textContent")[:59])
```

```
5 links with .//a; 47 with //a
''
[[ Guy 1 is talking to Guy 2, who is floating in the sky ]]
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: find comic 353's image three different ways: `By.ID` for the comic's `<div>` and then `By.TAG_NAME` inside it, one `By.CSS_SELECTOR`, and one short `By.XPATH` that starts from the id. Check with `==` that all three found the same element, and print its `alt`. **Share:** your three locators, whether `==` said `True`, and which one you'd trust after a redesign.

```
# Microsprint: find the comic's <img> with By.ID then By.TAG_NAME, with By.CSS_SELECTOR, and with
 ↪  By.XPATH
# Compare the three with ==, and print the image's alt attribute
by_id = driver.find_element(By.ID, "comic").find_element(By.TAG_NAME, "img")
by_css = driver.find_element(By.CSS_SELECTOR, "#comic img")
by_xpath = driver.find_element(By.XPATH, "//div[@id='comic']//img")
print(f"Same element: {by_id == by_css == by_xpath}; alt: {by_css.get_attribute('alt')}")
```

```
Same element: True; alt: Python
```

💬 Discuss

- Searching the first row with `//a` instead of `.//a` found every link on the page. What would a scraper with that bug have collected, and would it have raised an error?
- Selenium said the transcript is empty; `textContent` found it. Which answer is right for a study of what readers see, and which for a study of what the page contains?
- In week 5, the Inspector's **Copy XPath** described an element in one click. Which of your three locators would survive xkcd adding a banner above the comic, and which would a counted-down path like `/html/body/div[2]/div[2]/img` not?

Step 6 — Click and type (8 minutes)

xkcd's hover joke lives in the comic image's `title` attribute; its `alt` holds the comic's name (”Extracting Data”). Read both with `get_attribute()`. Then act like a reader: find the **Random** link with an XPath that matches part of its text with `contains()`, `.click()` it, and read the new comic.

Typing works the same way: find Wikipedia's search box by its `name`, `search`, type into it with `send_keys()`, and press Enter with `Keys.RETURN`. An exact title takes you straight to the article; anything else lands on a results page (Typing and Searching). Wikipedia welcomes automated access; a search engine like Google forbids it in its Terms of Service.

```
# Load https://xkcd.com; print the comic image's alt and title (XPath "//div[@id='comic']//img")
# Click the Random link (XPath "//ul[@class='comicNav']//a[contains(text(),'Random')]"), sleep 1
 ↪  second,
#   and print the new URL and its image's title
```

```
driver.get("https://xkcd.com")
img = driver.find_element(By.XPATH, "//div[@id='comic']//img")
print(f"{img.get_attribute('alt')}: {img.get_attribute('title')}")
driver.find_element(By.XPATH, "//ul[@class='comicNav']//a[contains(text(),'Random')]").click()
time.sleep(1)
img = driver.find_element(By.XPATH, "//div[@id='comic']//img")
print(f"{driver.current_url}: {img.get_attribute('title')}")
```

```
Vulnerability: Other numbers are largely unaffected, but anyone using '4' for their certificate
 ↪  key is urged to upgrade to '5'.
```

```
https://xkcd.com/1087/: My all-time favorite example of syntactic ambiguity comes from Wikipedia:
 ↪  'Charlotte's Web is a children's novel by American author E. B. White, about a pig named
 ↪  Wilbur who is saved from being slaughtered by an intelligent spider named Charlotte.'
```

```
# Import Keys; on https://www.wikipedia.org, type "Colorado Buffaloes" into the box named
 ↪  "search" and press Keys.RETURN
# Sleep 2 seconds, and print the page's title
from selenium.webdriver.common.keys import Keys

driver.get("https://www.wikipedia.org")
driver.find_element(By.NAME, "search").send_keys("Colorado Buffaloes", Keys.RETURN)
time.sleep(2)
print(driver.title)
```

```
Colorado Buffaloes - Wikipedia
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: a short version of Additional Exercise 1. Search Wikipedia for a topic of your own the same way, and count the results on the page you land on (`li.mw-search-result`): none means you landed on an article. **Share:** your query, the page's title, and how many results it listed.

```
# Microsprint: search Wikipedia for your own query the same way, and sleep 2 seconds
# Count the li.mw-search-result elements with find_elements, and print the page's title and the
 ↪  count
driver.get("https://www.wikipedia.org")
driver.find_element(By.NAME, "search").send_keys("headless browser scraping", Keys.RETURN)
time.sleep(2)
results = driver.find_elements(By.CSS_SELECTOR, "li.mw-search-result")
print(f"{driver.title}: {len(results)} results on this page")
```

```
headless browser scraping - Search results - Wikipedia: 8 results on this page
```

💬 Discuss

- Zero results meant you'd landed on an article. What else could give zero, and would your code notice the difference?
- Week 2 read `robots.txt` and Terms of Service as rules for scrapers. Do they still apply when a real browser does the requesting and a script does the clicking?
- `time.sleep(1)` after the click is a guess. What happens if the next comic takes three seconds to load?

Step 7 — Wait for what you need (7 minutes)

`time.sleep()` guesses how long a page needs. An **explicit wait** asks instead: `WebDriverWait` checks a condition every half second, returns as soon as it's true, and raises `TimeoutException` if it never is. Conditions come from `expected_conditions`, imported as `EC`: `presence_of_element_located`,

`visibility_of_element_located`, `element_to_be_clickable`, and `text_to_be_present_in_element` (Waiting for Dynamic Content).

The practice page `/js-delayed/` waits ten seconds before its JavaScript adds the quotes. Look after a two-second sleep, then wait properly, and time it.

```
# Import WebDriverWait and expected_conditions (as EC). Load
 ↪  https://quotes.toscrape.com/js-delayed/, timing from here
# Sleep 2 seconds and count the elements with class "quote"; then WebDriverWait(driver,
 ↪  15).until(EC.presence_of_element_located(...))
# Count them again, with the seconds since the page was requested
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

start = time.time()
driver.get("https://quotes.toscrape.com/js-delayed/")
time.sleep(2)
print(f"After a 2-second sleep: {len(driver.find_elements(By.CLASS_NAME, 'quote'))} quotes")
WebDriverWait(driver, 15).until(EC.presence_of_element_located((By.CLASS_NAME, "quote")))
print(f"After the wait, {time.time() - start:.1f} s in: {len(driver.find_elements(By.CLASS_NAME,
 ↪  'quote'))} quotes")
```

```
After a 2-second sleep: 0 quotes
After the wait, 10.3 s in: 10 quotes
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: a short version of Additional Exercise 4. Time two ways to load `https://quotes.toscrape.com/js/`: `requests.get()`, and `driver.get()` plus a wait for the first quote, in the browser that's already open. **Share:** your two times, and which was faster. We'll put the class's times side by side.

```
# Microsprint: time requests.get() with HEADERS on https://quotes.toscrape.com/js/,
#   then driver.get() plus a WebDriverWait for class "quote"; print both times
start = time.time()
requests.get("https://quotes.toscrape.com/js/", headers=HEADERS)
requests_seconds = time.time() - start
start = time.time()
driver.get("https://quotes.toscrape.com/js/")
WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.CLASS_NAME, "quote")))
print(f"requests.get(): {requests_seconds:.2f} s; the open browser: {time.time() - start:.2f} s")
```

```
requests.get(): 0.46 s; the open browser: 0.15 s
```

💬 Discuss

- A scraper that slept two seconds would have saved an empty page for every quote, with no error. Where else today could a fixed sleep have failed silently?
- The open browser loaded `/js/` faster than `requests.get()`: it keeps its connection open and caches the page's scripts. So where does a browser's cost come from? Look back at how long Step 4 took to start it.
- View Source on `/js-delayed/`: the quotes are in a script there too. Did this page need a browser at all?

Step 8 — Scroll to the end, then hand the page to BeautifulSoup (9 minutes)

On an infinite-scroll page, new content loads as you near the bottom. The chapter scrolls with `execute_script()`, which runs JavaScript in the page, and compares `document.body.scrollHeight` before and after each scroll: when the page stops growing, there's nothing more (Scraping Infinite Scroll

Pages). We write the lines of its `scrape_infinite_scroll()` straight, with its cap of 20 scrolls and its two-second pause.

Then hand `driver.page_source` to BeautifulSoup, which reads it all at once ("Passing to BeautifulSoup"): Selenium as the hands, BeautifulSoup as the eyes. When you have what you need, quit the browser: `driver.quit()` closes every window and stops chromedriver.

```
# Load https://quotes.toscrape.com/scroll and save last_height = driver.execute_script("return
 ↪  document.body.scrollHeight")
# Up to 20 times: run "window.scrollTo(0, document.body.scrollHeight);", sleep 2 seconds, read
 ↪  the height again,
#   and stop when it hasn't grown; print how many scrolls that took
driver.get("https://quotes.toscrape.com/scroll")
last_height = driver.execute_script("return document.body.scrollHeight")
for scroll in range(1, 21):
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
    time.sleep(2)
    new_height = driver.execute_script("return document.body.scrollHeight")
    if new_height == last_height:
        break
    last_height = new_height
print(f"No new content after scroll {scroll}; the page is {new_height}px tall")
```

```
No new content after scroll 10; the page is 15911px tall
```

```
# Hand driver.page_source to BeautifulSoup; make scroll_quotes, a DataFrame of author and text
 ↪  for each div.quote
# Print its shape, then driver.quit()
soup = BeautifulSoup(driver.page_source, "html.parser")
scroll_quotes = pd.DataFrame([
    {"author": q.find("small", class_="author").get_text(), "text": q.find("span",
 ↪  class_="text").get_text()}
    for q in soup.find_all("div", class_="quote")
])
print(scroll_quotes.shape)
driver.quit()
```

```
(100, 2)
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: check the scroll against Step 3. Do the scrolled page and the API give the same quotes? Compare the two sets of `text`, and the authors' counts. **Share:** how many texts are in both, how many in only one, and whether the authors' counts agree.

```
# Microsprint: compare set(scroll_quotes["text"]) with set(api_quotes["text"]): how many in both,
 ↪  and in only one?
# Do value_counts() of author agree between the two DataFrames?
in_scroll, in_api = set(scroll_quotes["text"]), set(api_quotes["text"])
print(f"{len(in_scroll & in_api)} in both; {len(in_scroll - in_api)} only scrolled; {len(in_api
 ↪  - in_scroll)} only from the API")
print(f"Authors' counts agree: {scroll_quotes['author'].value_counts().sort_index().
 ↪  equals(api_quotes['author'].value_counts().sort_index())}")
```

```
100 in both; 0 only scrolled; 0 only from the API
Authors' counts agree: True
```

💬 Discuss

- The browser and the API collected the same quotes. Which route would you defend in a methods section, and why?
- The chapter's `scrape_quotes()` quits its browser in a `try`/`finally` and checks each element for `None`; our cells do neither. What would break if a wait timed out, or the site renamed a class, and what would happen to the headless browser left running?
- `range(1, 21)` caps the scrolls at 20. On a feed with no end, how would you choose the cap, and what does week 2's proportionality question say about it?

Step 9 — Behind a login (9 minutes)

Quotes to Scrape has a practice login that accepts any username and password. Logged in, each quote shows a **(Goodreads page)** link that logged-out visitors don't see, so counting those links tells you whether you're logged in (”Pages Behind a Login”).

The chapter's first way is by hand: start a browser **with a window**, open the login page, and let `input("Log in in the Chrome window, then press Enter here")` hold the cell while you type, so that your password never enters your code. A headless browser has no window to type in, so here we use the chapter's second way, for code that must log in by itself: read the password from an environment variable, or ask for it with `getpass()`, which hides what you type and keeps it out of the notebook (Passwords and Other Credentials). On the practice site, type anything. Never print a password or a cookie's value; a cookie's name is safe.

```
# Read password from os.environ.get("QUOTES_PASSWORD") or getpass("Password: "); any password
 ↪  works on the practice site
# Start a new browser with options, and count the "(Goodreads page)" links (By.LINK_TEXT) on
 ↪  https://quotes.toscrape.com/
from getpass import getpass

password = os.environ.get("QUOTES_PASSWORD") or getpass("Password: ")
driver = webdriver.Chrome(options=options)
driver.get("https://quotes.toscrape.com/")
print(f"Logged out: {len(driver.find_elements(By.LINK_TEXT, '(Goodreads page)'))} Goodreads
 ↪  links")
```

```
Logged out: 0 Goodreads links
```

```
# On https://quotes.toscrape.com/login, type a username into #username, and the password into
 ↪  #password with Keys.RETURN
# Wait for the class "quote", then count the Goodreads links again; print the cookies' names
 ↪  (never their values), and quit
driver.get("https://quotes.toscrape.com/login")
driver.find_element(By.ID, "username").send_keys("your-username")
driver.find_element(By.ID, "password").send_keys(password, Keys.RETURN)
WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.CLASS_NAME, "quote")))
print(f"Logged in: {len(driver.find_elements(By.LINK_TEXT, '(Goodreads page)'))} Goodreads
 ↪  links")
print(f"Cookies: {[cookie['name'] for cookie in driver.get_cookies()]}")
driver.quit()
```

```
Logged in: 10 Goodreads links
Cookies: ['session']
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: a short version of Additional Exercise 6, on paper. Choose a site where you have an account and data you'd like to study. Without collecting anything, list the routes to it: an API and its price, a researcher program, data donation, asking the site, and a logged-in browser. For each, note who

carries the risk: you, the platform, or the people in the data. **Share:** your site, the route you'd take, and one risk that route puts on someone other than you.

```
# Microsprint: answer as comments. Your site, each route with its cost and who carries the risk,
 ↪  and the route you'd take
# Site: Reddit, to study how a city's subreddit talks about housing.
# API: week 3 quoted $0.24 per 1,000 calls for apps above the free limits; the risk is mostly the
 ↪  terms, on me.
# Researcher program: check whether Reddit runs one and what it requires; slow, but the platform
 ↪  shares the risk.
# Data donation: volunteers export their own data; consent is clean, but only the volunteers'
 ↪  posts, a biased sample.
# Asking: a message to the moderators and to Reddit; free, and may be ignored.
# Logged-in browser: free, but breaks terms I agreed to, stakes my account, and the posters never
 ↪  agreed to a dataset.
# I'd take the API, scoped to one subreddit and a fixed window, and tell an IRB the posts are
 ↪  public but the people didn't consent.
```

💬 Discuss

- Technically, logging in changed one cookie. What else changed: for you, for the site, and for the people whose posts you'd see?
- Week 3 put X's enterprise API at $42,000 a month and up. Is a logged-in browser the cheaper route, and who pays the difference?
- The chapter keeps a session in a Chrome profile folder so the next run can skip the login. Why does it say to treat that folder like your password, and where should it live?

Step 10 — The same with a Playwright script (6 minutes)

Playwright drives a browser too, with three differences: it installs its own browsers, its **locators** wait on their own when you act on or read an element, and it can record a script while you click (`python -m playwright codegen`). Its simple sync API won't run inside a notebook, because Jupyter already runs an event loop of its own ("Why Not in the Notebook?"). So write a script file, with `%%writefile` as the first line of the cell, and run it as a separate program with the notebook's own Python, as you would from a terminal.

The script puts two of the chapter's pieces together on `/scroll`: a locator that waits for the first quote's author ("Locators Wait for You"), and the API reply that scrolling triggers, caught as the page receives it (Catching the JSON Behind a Page): the request we watched by hand in Step 2. Playwright starts headless unless you pass `headless=False`. It needs its browsers first (`python -m playwright install chromium`, about 320 MB); in an older environment, add the library with `%conda install -y -c conda-forge playwright-python`.

```
%%writefile scroll_api.py
# A Playwright script: launch Chromium (pass headless=False to watch), open
 ↪  https://quotes.toscrape.com/scroll,
#   and print the first quote's author with page.locator("div.quote
 ↪  small.author").first.inner_text()
# Catch the "/api/quotes?page=2" reply with page.expect_response() while page.mouse.wheel(0,
 ↪  20000) scrolls; print its page and quotes
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch()
    page = browser.new_page()
    page.goto("https://quotes.toscrape.com/scroll")
    print(page.locator("div.quote small.author").first.inner_text())
    with page.expect_response(lambda r: "/api/quotes?page=2" in r.url) as info:
```

```
        page.mouse.wheel(0, 20000)
    data = info.value.json()
    print(f"Page {data['page']}: {len(data['quotes'])} quotes")
    browser.close()
```

```
Writing scroll_api.py
```

```
# Run the script with this notebook's Python, as a separate program: !{sys.executable}
 ↪  scroll_api.py
!{sys.executable} scroll_api.py
```

💬 **Discuss**

- Playwright's locator waited for the first quote without a `WebDriverWait`. Which of today's Selenium lines did the script not need?
- An AI agent connected through Playwright MCP could have collected these quotes for you. If it had, what could your methods section cite, compared with a script it wrote for you to read and rerun?
- Selenium drives Chrome, Firefox, Edge, or Safari as you have them installed; Playwright brings its own copies, pinned to its version. Which would you rather explain in a replication package, and why?

Step 11 — What did we learn? (4 minutes)

In a few sentences, as comments or in a Markdown cell:

- Of the checks in Steps 1–3, which found the quotes without a browser, and on which pages? How many requests and seconds did the API take, against the scroll's in Step 8?
- What did a browser cost to start in Step 4, and to load a page once it was open in Step 7's microsprint? For which page today was a browser the only way in?
- In Step 9, what told you that you were logged in, and what did logging in cost besides the code?
- Today's cells assumed every request and every wait would work. Which assumption would you guard first on a site of your own, and how does the chapter guard it?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, fetch your page statically and count your element: Step 1's `requests` and BeautifulSoup check, and week 2's `robots.txt`.
- Step 2, start a headless browser: Step 4's `options` and `webdriver.Chrome()`.
- Step 3, wait, then measure again: Step 7's `WebDriverWait`, and Step 8's `page_source` handed to BeautifulSoup.
- Step 4, load more: Step 8's scrolling loop, or Step 6's `.click()` for a **Load More** button.
- Step 5, tabulate and clean up: Step 3's DataFrame from a list of dictionaries, and Step 8's `driver.quit()`.
- Step 6, interpret: Steps 1–3's checks, and Step 7's timings.

```
# Your answers here, as comments
# Steps 1-3: requests found 0 div.quote elements on /js/, but its script held all 10 quotes; the
 ↪  hidden API gave all
#   100 quotes in 10 requests and 14.1 s. Step 8's browser scrolled 10 times, 2 s apart, to the
 ↪  same 100 (every text
#   matched), so at least 20 s.
# Step 4: starting Chrome took 16.2 s. Once open, it loaded /js/ in 0.15 s, faster than
 ↪  requests.get()'s 0.46 s, because
#   it reuses its connection and cache. No page today strictly needed a browser: each held its
 ↪  quotes in a script or
#   behind a JSON API. The login came closest.
# Step 9: 0 Goodreads links logged out, 10 logged in, and one cookie, "session". Besides the
 ↪  code, logging in costs
```

```
#   the terms you agreed to, your account, and the privacy of the people whose posts you'd see.
# Guard first: the hidden API. A 429 or an HTML error page would make response.json() fail in
 ↪  get_all_quotes(); the
#   chapter's loop calls raise_for_status() to stop on one. Then the browser: the chapter quits
 ↪  it in try/finally, so a
#   wait that times out doesn't leave a headless Chrome running.
```

Friday: from the lab to the textbook (15 minutes)

What did the lab turn up that chapter 8 could do better? Some places to look:

- **Outputs that changed.** xkcd's front page and the random comic change; did the chapter's xkcd numbers (2 navigation rows, 10 links) still hold when you ran Step 5? Did Step 4's browser version match the chapter's?
- **A cell that broke**, on your laptop and not on the instructor's screen: Selenium Manager, a missing driver, a window that wouldn't open. The chapter's "When Selenium Manager Fails" and "Common Issues to Debug" are where a fix would go.
- **A sentence that confused you.** The chapter's examples run on whatever page the browser happens to be on: which page is it on when "Scrolling" presses Page Down, or when "Passing to BeautifulSoup" counts links?
- **A microsprint that would make a good exercise**, such as checking the API's `top_ten_tags` against a tagged loop, or the three locators for one element.

See `handouts/common/revision-framework.md` for how to propose a revision.